PLANO DE ATIVIDADES

IDENTIFICAÇÃO

Curso: Engenharia de Software
Componente Curricular: Novas Aplicações em Engenharia de Software
Professor(a): Rafael Henrique Dalegrave Zottesso
Turma: 4º Ano

DEFINIÇÃO DO PROJETO

Desenvolver um projeto com Python e Django para a *Web*, de modo que contenha:

1. Pelo menos **seis classes** no *models* (sem contar a classe *User*)
2. Telas de inserir, alterar, excluir e listar para as classes do models, com exceção para casos que devem respeitar as regras de negócio
3. Um movimento implementado (não apenas o CRUD "básico")
4. Um *layout/template* com o Bootstrap, Tailwind, Bulma ou outro semelhante
    - Pode utilizar algum template pronto, desde que você tenha permissão para usá-lo (licença gratuita, etc)
5. Controle de acesso às páginas com a autenticação de usuários
6. Controle de acesso por grupos de usuários (ex: administrador, gerente, aluno, usuário) para mais de uma *view*
7. O uso de plugins/ferramentas com jQuery, Vue, React, Angular ou outra tecnologia *front-end* semelhante
8. Controle de versões com o Git (Github, Gitlab, Bitbucket ou similar)
    - https://www.udemy.com/course/git-e-github-para-iniciantes/

AVALIAÇÃO

O projeto será desenvolvido ao longo do ano e deve ser apresentado ao professor para validar o conteúdo e as avaliações

VIDEOAULAS

https://www.youtube.com/playlist?list=PLjv17QYEBJPpd6nI-MXpIa4qR7prKfPQz

APOSTILAS E CÓDIGOS

Django: Guia prático do iniciante
https://docs.google.com/document/d/13PMBiE3AwCjvp-iJ4obY8E1uPQcbIR3aUvbE2AAiVx8/

Projeto das videoaulas
https://github.com/rafaelzottesso/Curso-Django-2-Python-3

Conceito 1º Trimestre	
Prazo final de apresentação: 08/05 Recuperação: 15/05	
1º	• Criar um projeto Django (pode utilizar o que foi criado nas aulas) • Criar uma url, view e template para a página "sobre" contendo: ○ Descrição do seu projeto em forma de texto ○ Um **diagrama de caso de uso** (imagem) para apresentar as funcionalidades e atores do seu projeto (em forma de imagem no template) ○ Um **diagrama de classes** (imagem) para representar seu projeto (em forma de imagem no template) • Criar um template/layout fazendo o uso de algum *framework* Front End (ou utilizar algum pronto) • Criar classes e seus atributos baseados no diagrama de classe proposto • Reutilizar e personalizar os templates do Django (*extra_context* ou *def get_context_data*)
2º	• Com base no ***models***, criar as ***views***, ***urls*** e ***templates*** para: ○ inserir (CreateView) ○ alterar (UpdateView) ○ excluir (DeleteView) ○ detalhar (DetailView) ○ listar (ListView) • Instalar, ativar e aplicar o Django Crispy Forms (seu complemento para o *front-end* também) • Utilizar o banco de dados PostgreSQL da Neon.tech (***não*** *usar o sqlite*)
	Recursos e referências - Videoaulas 01 a 11 disponibilizadas no Youtube.

Conceito 2º Trimestre	
Entrega 1: 10/07/2026 Entrega 2: 21/08/2026 Recuperação: 28/08/2026 (entrega final)	
1º	• Criar os templates para todas as *views* do tipo *ListView* (com exceção daquelas que não precisam devido a regra de negócio do projeto) • Criar as *urls* para realizar: *login, logout e* alteração de senha • Controlar o acesso às *views* com autenticação do usuário (*LoginRequiredMixin* quando necessário conforme regras de negócio do projeto) • Utilizar o "*request.user.is_autenthicated*" para deixar o conteúdo dinâmico nos templates HTML (nos menus, por exemplo) • Relacionar e filtrar objetos por usuário utilizando os métodos *form_valid()* e *get_queryset()* nas *views* (quando a respectiva classe tiver relação com User de chave estrangeira): ○ inserir - "guardar" o *User* no atributo de chave estrangeira ○ editar - permitir que o *User* só edite seus registros ○ excluir - permitir que o *User* só exclua seus registros ○ listar - listar somente os objetos que o User cadastrou ○ detalhar - exibir somente o objeto que o User cadastrou
2º	• Utilizar as *QuerySets* do Django para fazer consultas e retornar objetos a outras *views* (últimas notícias, cadastros recentes, etc) • Controle de acesso às *views* por grupo de usuário, quando necessário (*GroupRequiredMixin*) (Vídeo #18) • Implementar a paginação nas *ListView* (*paginate_by*) e adicionar os *links* de paginação no template (Vídeo #35) • Ativar e utilizar o *Django Debug Toolbar* (Apostila pág 65) • Identificar *queries* (consultas) repetidas e otimizar com o *select_related* (Apostila pág 68) ou prefetch_related.

Conceito 3º Trimestre	
Prazo final de apresentação: 13/11 Recuperação: 04/12	
1º	1. Instalar o Django Debug Toolbar, identificar queries repetidas e otimizá-las com `select_related` (ou `prefetch_related`). 2. Utilizar o Django Filter em pelo menos duas `ListView` e implementar a paginação junto dos filtros por meio de uma `templatetag` própria que preserve os parâmetros da busca. 3. Associar os registros ao usuário que os cadastrou em pelo menos duas classes de modelo, permitindo que somente ele liste, visualize, altere e exclua seus registros. 4. Utilizar o framework de mensagens do Django em cadastro, alteração e exclusão. 5. Utilizar pelo menos dois plugins jQuery distintos e funcionais (DataTables, máscaras, galeria de fotos, calendário, Bootbox, etc.). 6. Fazer o deploy no Google App Engine e disponibilizar o link de acesso.
2º	1. Criar um `AGENTS.md` e pelo menos três skills para os agentes de IA, sendo uma delas para padronizar as mensagens de commit. 2. Criar o app `ia` como única porta de entrada para a IA, guardar no banco a configuração editável em tela, a auditoria de cada chamada e a cota mensal de tokens e ter as telas de requisições, configuração (com "Testar conexão"), IA por usuário e meu consumo. 3. Criar um assistente que usa `chamar_ia`, com o contexto do sistema em `contexto_projeto.md` e instruções com limites, guardar o histórico por usuário e enviar à IA só as últimas mensagens (janela de 5), enviar as perguntas por AJAX com CSRF, em uma tela no visual do site, com Markdown sanitizado. 4. Auditoria do código na plataforma TestAI. 5. Entregar o sistema funcional, com interface amigável e fluxo de navegação coerente.

SEQUÊNCIA DOS COMANDOS GIT

```
# Baixar o projeto no IFPR
git clone https://… (cole aqui a url do seu projeto)
# OU atualizar o projeto local caso já tenha no seu PC
git pull origin master
-----------------------------------------------------
# Configurar o usuário local somente uma vez
git config user.email "fulano@email.com"
git config user.name "Fulano de Tal"
-----------------------------------------------------
# 2 - Adicionar as alterações que você fez para poder registrá-las
git add *
# 3 - Registrar/efetuar as alterações em um novo commit (versão)
git commit -m “Mensagem do que foi feito”
# 4 - Enviar as alterações feitas para o projeto/repositório nas “nuvens”
# FAZ ISSO NO VSCODE MESMO
git push origin master
```